SE4020 - Mobile Application Development & Design - Assignment 2

Part B: VisionOS

Learning Reflections

Submitted by:

IT22346322 - MJM ARSHAQ

What I set out to learn

I had built flat iOS apps before, but never anything spatial. For this assignment I wanted to learn three things concretely:

1. **Spatial UX thinking when** a 3D *volume* genuinely beats a flat *window*, and how to make a spatial layout feel intentional rather than a UI panel floating in midair.
2. **Bridging SwiftUI and RealityKit** how a declarative SwiftUI app hosts a RealityKit scene, and how the two talk to each other (selection in, taps out, SwiftUI panels anchored *inside* the 3D scene).
3. **Why visionOS at all what** the platform gives you (windows, volumes, ornaments, spatial audio, hover, accessibility for entities) that a tablet cannot.

The prototype that came out of this is a Keyworker's "Spatial Day Dashboard": a roster window of five children, and a bounded nursery room volume where each child is a coloured peg you can tap to float out their day summary, rotate to walk around, with a spatial confirmation chime.

Curated resources and how each changed an implementation choice

1. WWDC23 - Get started with building apps for spatial computing

What it taught me: the three core scene types Window, **Volume**, and **Immersive Space and** that you should reach for the *least* immersive surface that solves the problem.

How it changed a choice: it directly shaped my surface split. The roster is a plain WindowGroup (flat, familiar, glanceable); the nursery room is a second WindowGroup with .windowStyle(.volumetric) and a defaultSize(... in: .meters) (see NurseryConnectVisionApp.swift). I consciously did **not** build an Immersive Space the session's "least immersive that works" framing convinced me a bounded volume gives the spatial value (walk around, depth, anchored panels) without the cost and risk of a full immersive scene.

2. WWDC23 - Meet SwiftUI for spatial computing

What it taught me: spatialspecific SwiftUI affordances .glassBackgroundEffect(), **ornaments**, and volumes get their size in realworld meters.

How it changed a choice: the floating control strip under the roster is an .ornament(attachmentAnchor: .scene(.bottom)) rather than a normal toolbar

(ContentView.swift), which is exactly the "controls that belong to the window but live outside its bounds" pattern from the session. The day panel uses .glassBackgroundEffect() so it is read as a spatial surface.

3. WWDC23 - Build spatial experiences with RealityKit + Apple's Hello World sample

What it taught me: building entities in code (ModelEntity, MeshResource.generate…, materials), making them interactive (InputTargetComponent, HoverEffectComponent, collision shapes), and crucially the **RealityView attachments** pattern for placing SwiftUI views *inside* a 3D scene.

How it changed a choice: this is the heart of the prototype. The whole room is built programmatically in NurseryRoomBuilder.swift (no Reality Composer Pro). The RealityView(content, attachments) { … } attachments: { … } form is *exactly* how ChildDayPanel anchors next to a tapped peg in NurseryRoomView.swift the panel is a SwiftUI Attachment (id:) whose anchor entity I position relative to the selected peg each update. Tapping resolves the child by walking up from the hit entity to the one carrying my custom ChildTagComponent.

4. WWDC24 - What's new in visionOS

What it taught me: refinements to hover styling and accessibility for RealityKit entities.

How it changed a choice: pegs use the styled HoverEffectComponent(.highlight(…)) rather than the default, and each peg carries an AccessibilityComponent (label = name + room, value = hint,. button trait) so VoiceOver announces it both in NurseryRoomBuilder.swift.

What I deliberately did not use, and why

- **ImmersiveSpace** out of scope by choice. A bounded volume already delivers the spatial payoff; full immersion added complexity and review risk for no extra marks against this brief.
- **Reality Composer Pro / USDZ assets built** the scene in code instead. It keeps the repo selfcontained, makes the layout reviewable as Swift, and avoids binary asset tooling.
- **SwiftData / CloudKit** the store is an inmemory @Observable seeded with five children. Persistence wasn't needed to demonstrate the spatial interactions and would have diluted focus.

- **Handtracking / custom gestures** SpatialTapGesture + DragGesture cover the three required interactions cleanly; bespoke hand tracking was unjustified for the scope.

What I'd do with more time

- **SwiftData persistence** so diary entries and incidents survive launches.
- **An ambient room bed** (low nursery murmur) layered under the tap chime for presence.
- **Richer assets real** locker/cubby models and child avatars instead of coloured boxes.
- **An optional Immersive Space** "room walkthrough" mode as a stretch surface.
- **More automated coverage** of the entity resolution logic (the tap → ChildTagComponent walk).

The most valuable lesson was the surface decision itself: spatial computing isn't "3Dify the UI," it's choosing *which* parts earn depth. The roster stayed flat on purpose; only the room where walking around and anchoring information in space adds real meaning became a volume.